

Principles of Computer Science II

Dynamic Programming and Sequence Similarity

Marco Zecchini

Sapienza University of Rome

Lecture 9

Coin Change Problem

United States Change Problem:

Convert some amount of money into the fewest number of coins.

Input: An amount of money, M , in cents.

Output: The smallest number of quarters q , dimes d , nickels n , and pennies p whose values add to M (i.e., $25q + 10d + 5n + p = M$ and $q + d + n + p$ is as small as possible).

- We know that the greedy solution is not good..
- ... we are only left with a brute-force approach that is impractical

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;
- the best combination for $77 - 3 = 74$ cents, plus a 3-cent coin;

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;
- the best combination for $77 - 3 = 74$ cents, plus a 3-cent coin;
- the best combination for $77 - 7 = 70$ cents, plus a 7-cent coin.

Finding the best combination

Suppose you need to make change for 77 cents and the only coin denominations available are 1, 3, and 7 cents.

The best combination for 77 cents will be one of the following:

- the best combination for $77 - 1 = 76$ cents, plus a 1-cent coin;
- the best combination for $77 - 3 = 74$ cents, plus a 3-cent coin;
- the best combination for $77 - 7 = 70$ cents, plus a 7-cent coin.

$$bestNumCoins_M = \min \begin{cases} bestNumCoins_{M-1} + 1 \\ bestNumCoins_{M-3} + 1 \\ bestNumCoins_{M-7} + 1 \end{cases}$$

Same for 76 and, then, 75...

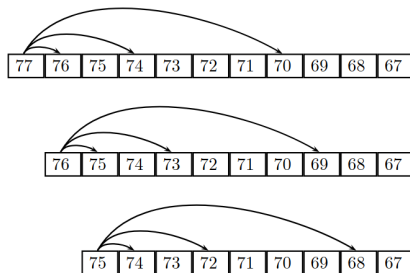


Figure 6.1 The relationships between optimal solutions in the Change problem. The smallest number of coins for 77 cents depends on the smallest number of coins for 76, 74, and 70 cents; the smallest number of coins for 76 cents depends on the smallest number of coins for 75, 73, and 69 cents, and so on.

RecursiveChange

```
RECURSIVECHANGE( $M, \mathbf{c}, d$ )
1  if  $M = 0$ 
2      return 0
3   $bestNumCoins \leftarrow \infty$ 
4  for  $i \leftarrow 1$  to  $d$ 
5      if  $M \geq c_i$ 
6           $numCoins \leftarrow \text{RECURSIVECHANGE}(M - c_i, \mathbf{c}, d)$ 
7          if  $numCoins + 1 < bestNumCoins$ 
8               $bestNumCoins \leftarrow numCoins + 1$ 
9  return  $bestNumCoins$ 
```


RecursiveChange

```
RECURSIVECHANGE( $M, \mathbf{c}, d$ )
1  if  $M = 0$ 
2      return 0
3   $bestNumCoins \leftarrow \infty$ 
4  for  $i \leftarrow 1$  to  $d$ 
5      if  $M \geq c_i$ 
6           $numCoins \leftarrow \text{RECURSIVECHANGE}(M - c_i, \mathbf{c}, d)$ 
7          if  $numCoins + 1 < bestNumCoins$ 
8               $bestNumCoins \leftarrow numCoins + 1$ 
9  return  $bestNumCoins$ 
```

- However, it computes the optimum amount of coin repeatedly

RecursiveChange

```
RECURSIVECHANGE( $M, \mathbf{c}, d$ )
1  if  $M = 0$ 
2      return 0
3   $bestNumCoins \leftarrow \infty$ 
4  for  $i \leftarrow 1$  to  $d$ 
5      if  $M \geq c_i$ 
6           $numCoins \leftarrow \text{RECURSIVECHANGE}(M - c_i, \mathbf{c}, d)$ 
7          if  $numCoins + 1 < bestNumCoins$ 
8               $bestNumCoins \leftarrow numCoins + 1$ 
9  return  $bestNumCoins$ 
```

- However, it computes the optimum amount of coin repeatedly
- The optimal coin combination for 70 cents is recomputed repeatedly nine times over and over as
(77 - 7), (77 - 3 - 3 - 1), (77 - 3 - 1 - 3), (77 - 1 - 3 - 3), (77 - 3 - 1 - 1 - 1 - 1), (77 - 1 - 3 - 1 - 1 - 1), (77 - 1 - 1 - 3 - 1 - 1), (77 - 1 - 1 - 1 - 3 - 1), (77 - 1 - 1 - 1 - 1 - 3) and (77 - 1 - 1 - 1 - 1 - 1 - 1).

Reverse the order...

Leverage previously computed solutions to form solutions to larger problems and avoid all this recomputation

```
DPCHANGE( $M, c, d$ )
1   $bestNumCoins_0 \leftarrow 0$ 
2  for  $m \leftarrow 1$  to  $M$ 
3       $bestNumCoins_m \leftarrow \infty$ 
4      for  $i \leftarrow 1$  to  $d$ 
5          if  $m \geq c_i$ 
6              if  $bestNumCoins_{m-c_i} + 1 < bestNumCoins_m$ 
7                   $bestNumCoins_m \leftarrow bestNumCoins_{m-c_i} + 1$ 
8  return  $bestNumCoins_M$ 
```

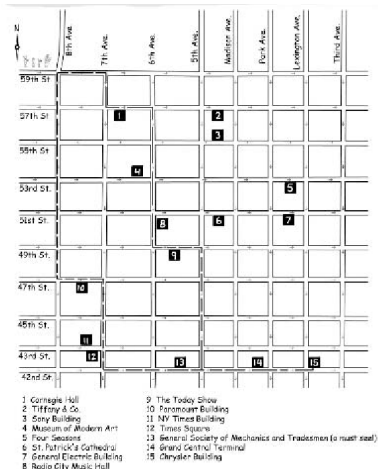
We compute a solution for each possible amount of money m from 1 to M and, since it takes d steps, to find the right coin the cost is $\mathcal{O}(Md)$.

Dynamic Programming

- Dynamic programming solves problems by combining the solutions to subproblems.
- “Programming” refers to a tabular method, not to writing computer code.
- Dynamic programming applies when the subproblems overlaps - that is, when subproblems share subsubproblems
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem

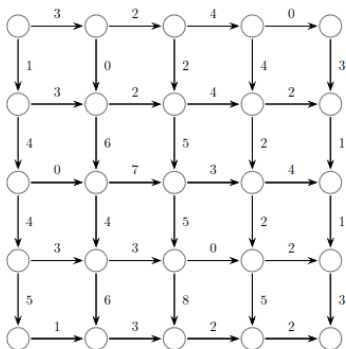
Manhattan Tourist Problem

- Sightseeing tour in Manhattan where a group of tourists wants to walk from the corner of 59th Street and 8th Avenue to the Chrysler Building.
- Many attractions along the way and the tourists want to see as many attractions as possible.
- The tourists can move either to the south or to the east, but even so, they can choose from many different paths.



Manhattan Tourist Problem as a graph

- We can represent this gridlike structure as a *graph*
- Intersections are *vertexes*
- Streets are *edges* that are oriented either towards south ($\downarrow$) or east ($\rightarrow$) and have a *weight*
- A *path* is a continuous sequence of edges, and the *length of a path* is the sum of the edge weights in the path



Problem statement

Manhattan Tourist Problem:

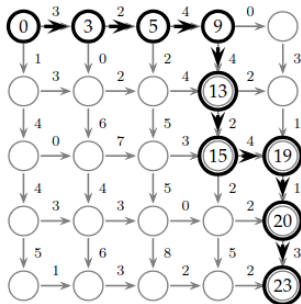
Find a longest path in a weighted grid.

Input: A weighted grid G with two distinguished vertices:
a source and a sink.

Output: A longest path in G from source to sink.

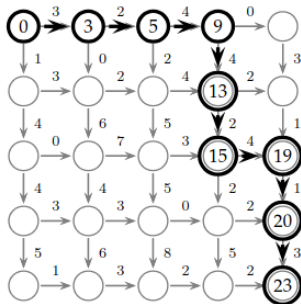
Greedy approach

- Always pick the maximum edge for each vertex



Greedy approach

- Always pick the maximum edge for each vertex
- Can we do better?

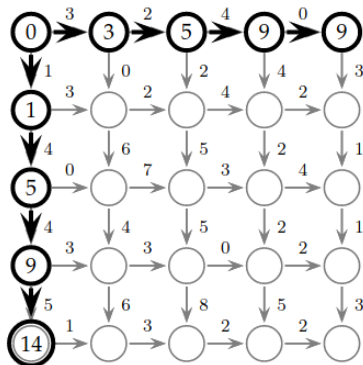


DP Approach: easier subproblems

- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.

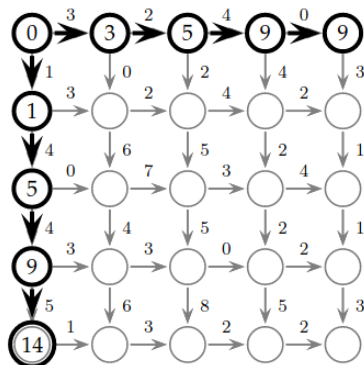
DP Approach: easier subproblems

- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.
- Let $s_{i,j}$ denotes the optimal solution for the vertex (i, j) .



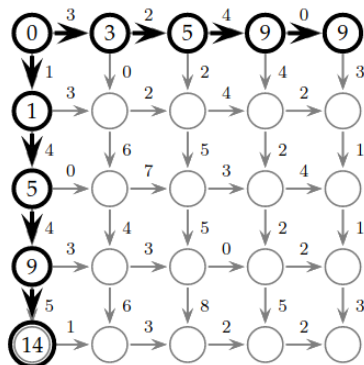
DP Approach: easier subproblems

- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.
- Let $s_{i,j}$ denotes the optimal solution for the vertex (i, j) .
- Finding $s_{0,j}$ (for $0 \leq j \leq m$) is not hard because tourists cannot choose an arbitrary path: they can only go east.

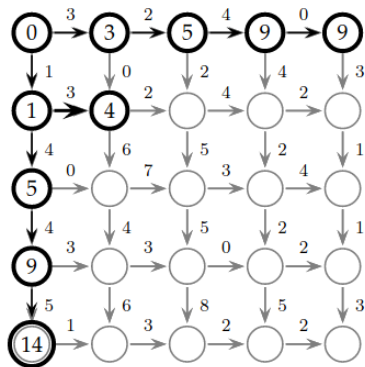


DP Approach: easier subproblems

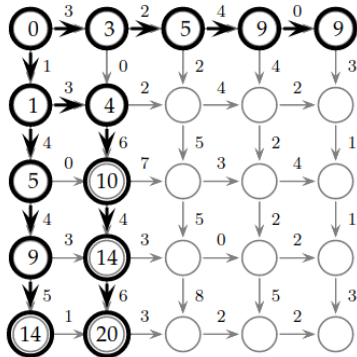
- Let us solve a more general problem: find the longest path from source to an arbitrary vertex (i, j) with $0 \leq i \leq n, 0 \leq j \leq m$.
- Let $s_{i,j}$ denotes the optimal solution for the vertex (i, j) .
- Finding $s_{0,j}$ (for $0 \leq j \leq m$) is not hard because tourists cannot choose an arbitrary path: they can only go east.
- The same for $s_{i,0}$ (for $0 \leq i \leq n$).



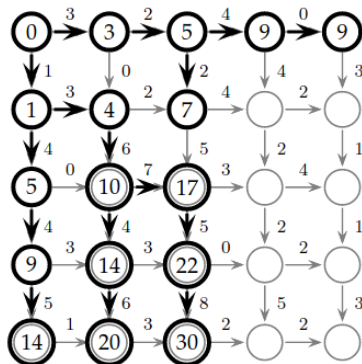
- Let us find the solution for $s_{1,1}$.
- Tourist can arrive to $(1, 1)$ in only two ways: from $(0, 1)$ or from $(1, 0)$
- The best path will be: either (1) $s_{0,1} +$ the weight from $(0, 1)$ to $(1, 1)$ or (2) $s_{1,0} +$ the weight from $(1, 0)$ to $(1, 1)$.
- We take the largest value.



- Similar logic to $s_{2,1}$, $s_{3,1}$ and so on.

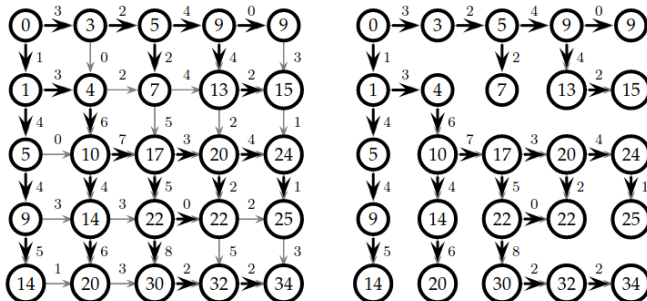


- Let us find the solution for $s_{1,2}$.
- Tourist can arrive to $(1, 2)$ in only two ways: from $(1, 1)$ or from $(2, 0)$
- The best path will be: either (1) $s_{1,1}$ + the weight from $(1, 1)$ to $(1, 2)$ or (2) $s_{0,2}$ + the weight from $(0, 2)$ to $(1, 2)$.
- We take the largest value.
- Similar logic to $s_{2,2}$, $s_{3,2}$ and so on.



We can compute the optimal solution for each vertex with this approach.

$$s_{i,j} = \max \left\{ \begin{array}{l} s_{i-1,j} + \text{weight of the edge between } (i-1,j) \text{ and } (i,j) \\ s_{i,j-1} + \text{weight of the edge between } (i,j-1) \text{ and } (i,j) \end{array} \right.$$



DP Approach: the algorithm

- $\downarrow$ $\mathbf{w}$ two-dimensional array representing the weights of the grid's edges that run north to south
- $\rightarrow$ $\mathbf{w}$ is a two-dimensional array representing the weights of the grid's edges that run west to east.

```

MANHATTANTOURIST( $\overleftarrow{\mathbf{w}}, \overrightarrow{\mathbf{w}}, n, m$ )
1   $s_{0,0} \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $n$ 
3       $s_{i,0} \leftarrow s_{i-1,0} + \overleftarrow{w}_{i,0}$ 
4  for  $j \leftarrow 1$  to  $m$ 
5       $s_{0,j} \leftarrow s_{0,j-1} + \overrightarrow{w}_{0,j}$ 
6  for  $i \leftarrow 1$  to  $n$ 
7      for  $j \leftarrow 1$  to  $m$ 
8           $s_{i,j} \leftarrow \max \begin{cases} s_{i-1,j} + \overleftarrow{w}_{i,j} \\ s_{i,j-1} + \overrightarrow{w}_{i,j} \end{cases}$ 
9  return  $s_{n,m}$ 

```

Initial Conditions

Recurrence

DP Approach: the algorithm

- $\vec{w}$ two-dimensional array representing the weights of the grid's edges that run north to south
- $\overleftarrow{w}$ is a two-dimensional array representing the weights of the grid's edges that run west to east.
- The cost is $\mathcal{O}(nm)$

```

MANHATTANTOURIST( $\vec{w}, \overleftarrow{w}, n, m$ )
1   $s_{0,0} \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $n$ 
3       $s_{i,0} \leftarrow s_{i-1,0} + \vec{w}_{i,0}$ 
4  for  $j \leftarrow 1$  to  $m$ 
5       $s_{0,j} \leftarrow s_{0,j-1} + \overleftarrow{w}_{0,j}$ 
6  for  $i \leftarrow 1$  to  $n$ 
7      for  $j \leftarrow 1$  to  $m$ 
8           $s_{i,j} \leftarrow \max \begin{cases} s_{i-1,j} + \vec{w}_{i,j} \\ s_{i,j-1} + \overleftarrow{w}_{i,j} \end{cases}$ 
9  return  $s_{n,m}$ 

```

Initial Conditions

Recurrence

Steps to design a DP algorithm.

Dynamic programming typically applies to *optimization problems*: each solution has a value (i.e., it is a number) and you want to find a solution with the optimal (minimum or maximum) value. To develop a dynamic-programming algorithm, follow a sequence of four steps:

- ① Characterize the structure of an optimal solution.
- ② Recursively define the value of an optimal solution.
- ③ Compute the value of an optimal solution, typically in a bottom-up fashion.
- ④ Construct an optimal solution from computed information.

When do we use DP in
bioinformatics?

Sequence Similarity

- We looked for repeating patterns within DNA sequences.
- How can we measure the similarity between different sequences?

Similarity of ATATATAT vs TATATATA

A	T	A	T	A	T	A	T
:	:	:	:	:	:	:	:
T	A	T	A	T	A	T	A

Hamming distance

[31 languages](#)[Article](#) [Talk](#)[Read](#) [Edit](#) [View history](#) [Tools](#)

From Wikipedia, the free encyclopedia



This article includes a list of [general references](#), but it **lacks sufficient corresponding inline citations**. Please help to [improve](#) this article by [introducing](#) more precise citations. *(May 2015)* ([Learn how and when to remove this message](#))

In [information theory](#), the **Hamming distance** between two [strings](#) or vectors of equal length is the number of positions at which the corresponding [symbols](#) are different. In other words, it measures the minimum number of *substitutions*

Hamming distance



Sequence Similarity

- We looked for repeating patterns within DNA sequences.
- How can we measure the similarity between different sequences?

Similarity of ATATATAT vs TATATATA

A	T	A	T	A	T	A	T
:	:	:	:	:	:	:	:
T	A	T	A	T	A	T	A

Alignment of ATATATAT vs TATATATA

A	T	A	T	A	T	A	T	-
:	:	:	:	:	:	:	:	:
-	T	A	T	A	T	A	T	A

Edit Distance

- We use the notion of Vladimir Levenshtein introduced in 1966
- **Edit distance** – the **minimum** number of editing operations needed to transform one string into another (insert/delete symbol or substitute one symbol for another).

Alignment of ATATATAT vs TATAAT

A	T	A	T	A	T	A	T
	:	:	:	:	:	:	:
-	T	A	T	A	-	A	T

Edit Distance

Alignment of TGCATAT vs ATCCGAT

TGCATAT

↓

delete last T

TGCATA

↓

delete last A

TGCAT

↓

insert A at the front

ATGCAT

↓

substitute C for G in the third position

ATCCAT

↓

insert a G before the last A

ATCCGAT

Five operations.

Edit Distance

Alignment of TGCATAT vs ATCCGAT

TGCATAT



insert A at the front

ATGCATAT



delete T in the sixth position

ATGCAAT



substitute G for A in the fifth position

ATGCGAT



substitute C for G in the third position

ATCCGAT

Four operations.

Edit Distance

- Vladimir Levenshtein defined the notion of **Edit distance**
- Did not provide an algorithm to compute it.

Edit Distance Algorithm using Dynamic Programming

- Assume two strings:
 - v (of n characters)
 - w (of m characters)
- The alignment of v, w is a two-row matrix such that
 - first row: contains the characters of v (in order)
 - second row: contains the characters of w (in order)
 - spaces are interspersed throughout the table, *no replaces*
- Characters in each string appear in order, though not necessarily adjacently.

A	T	-	G	T	T	A	T	-
A	T	C	G	T	-	A	-	C

- No column contains spaces in both rows.
- At most $n + m$ columns.

Edit Distance Algorithm using Dynamic Programming

A	T	-	G	T	T	A	T	-
A	T	C	G	T	-	A	-	C

- **Matches** – columns with the same letter,
- **Mismatches** – columns with different letters.
- Columns containing one space are called **indels**
 - Space on top row: **insertions**
 - Space on bottom row: **deletions**

$$\# \text{ matches} + \# \text{ mismatches} + \# \text{ indels} < n + m$$

Representing the rows

v	A	T	-	G	T	T	A	T	-
w	A	T	C	G	T	-	A	-	C

- One way to represent v
 - AT-CGTAT-
- One way to represent w
 - ATCGT-A-C
- Another way to represent v
 - AT-CGTAT-
 - 122345677
 - number of symbols of v present up to a given position
- Similarly, to represent w
 - ATCGT-A-C
 - 123455667

Representing the rows

v	A	T	-	G	T	T	A	T	-
w	A	T	C	G	T	-	A	-	C

v	1	2	2	3	4	5	6	7	7
w	1	2	3	4	5	5	6	6	7

can be viewed as a coordinate in 2-dimensional $n \times m$ grid:

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} \begin{pmatrix} 4 \\ 5 \end{pmatrix} \begin{pmatrix} 5 \\ 5 \end{pmatrix} \begin{pmatrix} 6 \\ 6 \end{pmatrix} \begin{pmatrix} 7 \\ 6 \end{pmatrix} \begin{pmatrix} 7 \\ 7 \end{pmatrix}$$

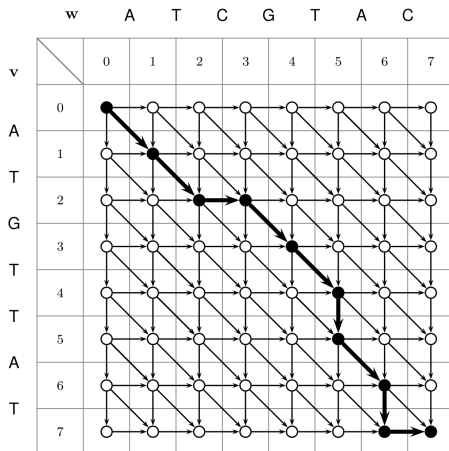
The entire alignment is simply a path:

$$(0, 0) \rightarrow (1, 1) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 4) \rightarrow (4, 5) \rightarrow (5, 5) \rightarrow (6, 6) \rightarrow (7, 6) \rightarrow (7, 7)$$

Edit distance graph

- **Edit graph**: a grid of n, m size.
- The edit graph will help us in calculating the edit distance.
- Alignment: a **path** from $(0, 0)$ to (n, m) .
- Every alignment corresponds to a **path** in the edit graph.
- Diagonal movement at point i, j correspond to column $\begin{pmatrix} v_i \\ w_j \end{pmatrix}$
- Horizontal movement correspond to column $\begin{pmatrix} - \\ w_j \end{pmatrix}$
- Vertical movement correspond to column $\begin{pmatrix} v_i \\ - \end{pmatrix}$

Edit distance graph



$\swarrow$ $\searrow$ $\rightarrow$ $\swarrow$ $\searrow$ $\downarrow$ $\swarrow$ $\downarrow$ $\rightarrow$
 A T - G T T A T -
 A T C G T - A - C

Scoring the alignment

- Given any two strings, there are many different alignment matrices and corresponding paths in the edit graph.
- Some of these have a surplus of mismatches and indels and a few matches, while others have many matches and few indels and mismatches.

Scoring the alignment

- Given any two strings, there are many different alignment matrices and corresponding paths in the edit graph.
- Some of these have a surplus of mismatches and indels and a few matches, while others have many matches and few indels and mismatches.
- We introduce the notion of a **scoring** function, which takes as input an alignment matrix (a path in the edit graph) and produces a score that determines the “goodness” of the alignment.

Scoring the alignment

- Given any two strings, there are many different alignment matrices and corresponding paths in the edit graph.
- Some of these have a surplus of mismatches and indels and a few matches, while others have many matches and few indels and mismatches.
- We introduce the notion of a **scoring** function, which takes as input an alignment matrix (a path in the edit graph) and produces a score that determines the “goodness” of the alignment.
- *Example:* +1 if in v_i and w_i same letter, 0 otherwise. Score is the sum of the column scores.

Scoring the alignment

- Given any two strings, there are many different alignment matrices and corresponding paths in the edit graph.
- Some of these have a surplus of mismatches and indels and a few matches, while others have many matches and few indels and mismatches.
- We introduce the notion of a **scoring** function, which takes as input an alignment matrix (a path in the edit graph) and produces a score that determines the “goodness” of the alignment.
- *Example:* +1 if in v_i and w_i same letter, 0 otherwise. Score is the sum of the column scores.
- By choosing different scoring functions, we can solve different string comparison problems.

Longest Common Subsequence (LCS)

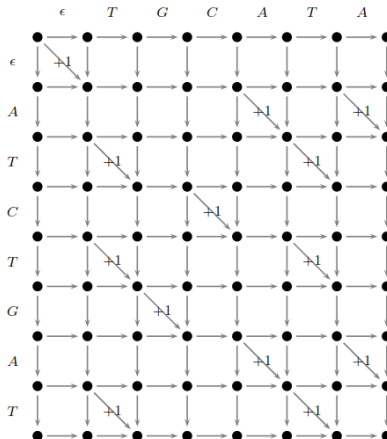
- Similar problem to Edit Distance
- A *subsequence* is an ordered sequence of characters (not necessarily, consecutive).
- For ATTGCTA, AGCA is a subsequence, TGTT is not.
- A subsequence is *common* to two strings if it is a subseq of them both
- TCTA is a common to both **ATCTGAT** and **TGCATA**

Longest Common Subsequence Problem:

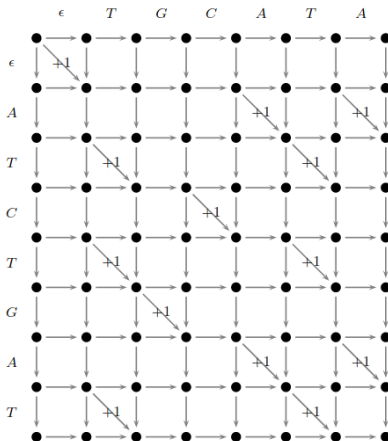
Find the longest subsequence common to two strings.

Input: Two strings, v and w .

Output: The longest common subsequence of v and w .



LCS Edit Graph



Similar to Manhattan Tourist problem

Recurrence in LCS

- Let $s_{i,j}$ be the length of LCS for the i -prefix of v_i and j -prefix of w_j

Recurrence in LCS

- Let $s_{i,j}$ be the length of LCS for the i -prefix of v_i and j -prefix of w_i
- $s_{i,0} = s_{0,j} = 0$ for all $1 \leq i \leq n$ and $1 \leq j \leq m$

Recurrence in LCS

- Let $s_{i,j}$ be the length of LCS for the i -prefix of v_i and j -prefix of w_i
- $s_{i,0} = s_{0,j} = 0$ for all $1 \leq i \leq n$ and $1 \leq j \leq m$
- $s_{i,j}$ satisfies the following recurrence:

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + 0 \\ s_{i,j-1} + 0 \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \end{cases}$$

LCS Algorithm

```
LCS(v, w)
1  for i ← 0 to n
2      si,0 ← 0
3  for j ← 1 to m
4      s0,j ← 0
5  for i ← 1 to n
6      for j ← 1 to m
7          si,j ← max {  $\begin{cases} s_{i-1,j} \\ s_{i,j-1} \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \end{cases}$ 
8  return (sn,m, b)
```

Edit Distance Recurrence

- Let $d_{i,j}$ be the number of transformations for the i -prefix of v_i and j -prefix of w_j
- $d_{i,0} = i$ and $s_{0,j} = j$ for all $1 \leq i \leq n$ and $1 \leq j \leq m$
- $s_{i,j}$ satisfies the following recurrence:

$$s_{i,j} = \min \begin{cases} s_{i-1,j} + 1 \\ s_{i,j-1} + 1 \\ s_{i-1,j-1}, & \text{if } v_i = w_j \end{cases}$$

Edit Distance Python program

```
def edit_distance(s1, s2):
    m=len(s1)+1
    n=len(s2)+1

    tbl = {}
    for i in range(m): tbl[i,0]=i
    for j in range(n): tbl[0,j]=j
    for i in range(1, m):
        for j in range(1, n):
            cost = 0 if s1[i-1] == s2[j-1] else 1
            tbl[i,j] = min(tbl[i, j-1]+1,
                           tbl[i-1, j]+1,
                           tbl[i-1, j-1]+cost)

    return tbl[i,j]
```

Global Sequence Alignment

- The LCS problem corresponds to a rather restrictive scoring that awards 1 for matches and does not penalize indels.

Global Sequence Alignment

- The LCS problem corresponds to a rather restrictive scoring that awards 1 for matches and does not penalize indels.
- To generalize scoring, we extend the k -letter alphabet to include the gap character “-”, and consider an arbitrary $(k + 1) \times (k + 1)$ scoring matrix δ .

Global Sequence Alignment

- The LCS problem corresponds to a rather restrictive scoring that awards 1 for matches and does not penalize indels.
- To generalize scoring, we extend the k -letter alphabet to include the gap character “-”, and consider an arbitrary $(k + 1) \times (k + 1)$ scoring matrix δ .
- The score of the column (x, y) in the alignment is $\delta(x, y)$ and the alignment score is defined as the sum of the scores of the columns. The recurrence will be:

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \delta(v_i, -) \\ s_{i,j-1} + \delta(-, w_j) \\ s_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$